
Ensemble Methods: Boosting vs. Bagging

An Empirical Study

Shahin Alakparov Narmina Ibragimova Hasan Mammadov

AI Academy, National AI Center - Summer 2026

Motivation & Central Question

- Ensemble learning combines weak learners to reduce generalisation error beyond any single model.
- Two dominant paradigms exist with *fundamentally different* error-reduction strategies:
 - **Boosting** - sequential, additive model that *targets bias*: each round re-weights misclassified examples.
 - **Bagging** - parallel, independent trees that *target variance*: averaging decorrelated hypotheses.

Central question: *Under what conditions does boosting outperform bagging, and vice versa - and why?*

- All algorithms implemented **from scratch in NumPy**: CART, AdaBoost (SAMME), and Random Forest.
- Scikit-learn serves only as a correctness baseline, *not* as a model source.

Datasets

Dataset	N	P	Classes	Key Challenge
Breast Cancer Wisconsin	569	30	2	Small, low-noise, well-separated classes
Adult Income (Census)	45 222	100	2	Large, class imbalance ($\approx 76\%$ negative)
MNIST 3-vs-8	6 000	784	2	High-dimensional, $56\times$ more features than samples per class

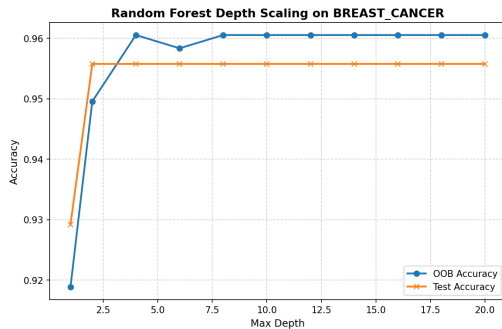
- The three datasets span orthogonal difficulty axes: *size*, *dimensionality*, and *class balance* - allowing conclusions to generalise beyond a single regime.
- All experiments use fixed seeds for reproducibility.

Method 1: Decision Tree (CART)

- Recursive binary splits: choose the threshold that maximises the *information gain*:

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

- Naive search is $O(N^2P)$; we replace it with a single sort + `np.cumsum` sweep: $O(N \log N \cdot P)$.
- Weighted Gini/entropy impurity: critical for AdaBoost where each sample carries a different weight.
- Stopping criteria: max depth, min leaf size, zero impurity gain, all-identical features.



Accuracy plateaus after depth ≈ 4 on Breast Cancer - deeper trees gain no purity but increase variance.

Method 2: AdaBoost (Discrete SAMME)

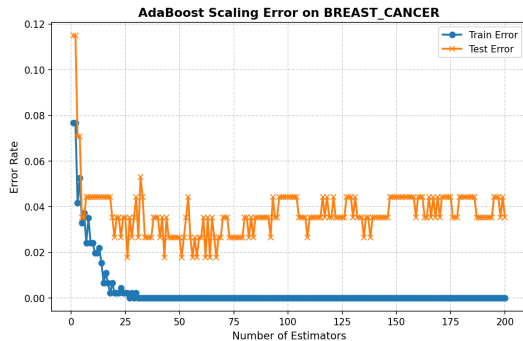
At each round m : fit a depth-1 stump h_m on the current weight distribution, then

$$\epsilon_m = \sum_i w_i^{(m)} 1[h_m(x_i) \neq y_i]$$

$$\alpha_m = \ln \frac{1 - \epsilon_m}{\epsilon_m} + \ln(K - 1)$$

$$w_i^{(m+1)} \propto w_i^{(m)} \exp(\alpha_m \cdot 1[h_m(x_i) \neq y_i])$$

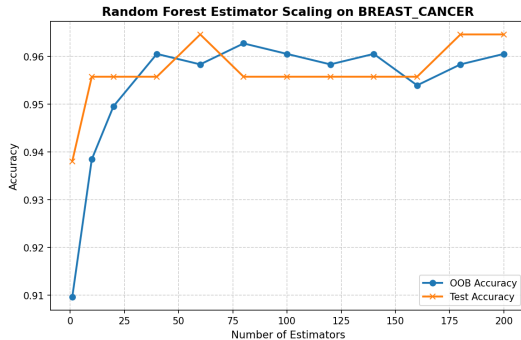
Early-stop when $\epsilon_m \geq (K - 1)/K$ (stump is no better than chance).



Train/test accuracy vs. number of stumps. Test accuracy saturates around 50 stumps - Exp 2 determines the optimal ensemble size across datasets.

Method 3: Random Forest

- Each tree trained on a *bootstrap* sample of size N (with replacement); OOB samples form a free validation set.
- At each node, only $\lfloor \sqrt{P} \rfloor$ randomly selected features are considered - this *decorrelates* trees, which is the mechanism that reduces variance.
- Trees grown in parallel via multiprocessing.Pool.
- Final prediction: majority vote (classification); OOB score is an unbiased accuracy estimate without a hold-out split.



Accuracy vs. number of trees (Breast Cancer). OOB score converges near $T = 60$, demonstrating variance reduction saturates quickly.

Experimental Design Overview

- **Exp. 1** *Correctness verification*: custom tree vs. sklearn on a fixed 80/20 split. Requirement: $|\Delta_{\text{acc}}| \leq 2\%$.
- **Exp. 2** *AdaBoost scaling*: test accuracy vs. #stumps from 1 to 200. Identifies saturation and over-fitting regime.
- **Exp. 3** *Random Forest scaling*: (a) accuracy vs. #trees; (b) accuracy vs. max depth. Examines variance reduction convergence.
- **Exp. 4** *Head-to-head*: stratified 5-fold CV, 100 estimators - fair multi-metric comparison (Accuracy, Macro-F1, AUC-ROC).
- **Exp. 5** *Noise robustness*: random label flips at $\eta \in \{0, 0.05, 0.10, 0.20\}$.
- **Exp. 6** *Bias-variance decomposition*: $B=100$ bootstrap replicates of training, Breast Cancer dataset.
- **Exp. 7** *Unsupervised structure*: PCA scree, K-Means elbow, DBSCAN k -distance, PCA scatter.
- **Bonus**: SAMME vs. SAMME.R calibration; GBM vs. AdaBoost.

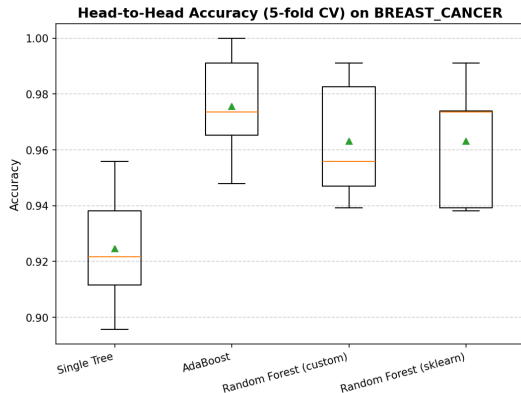
Exp. 1 - Correctness: Custom Tree vs. Sklearn

Why this matters:

A from-scratch implementation risks subtle numerical deviations in split search, class probability estimation, or threshold midpoint computation. Agreement within 2% provides empirical evidence that our tree is algorithmically equivalent.

Dataset	Custom	Sklearn	$ \Delta $
Breast Cancer	0.9469	0.9469	0.0000
Adult Income	0.8092	0.8105	0.0013
MNIST 3-vs-8	0.9292	0.9317	0.0025

Interpretation: All differences $\leq 0.0025 \ll 0.02$ threshold. The marginal Adult/MNIST gaps arise from tie-breaking order in split search - not from algorithmic error.



5-fold CV (Exp 4) confirms the custom tree generalises comparably to sklearn across all folds, with ensemble models substantially outperforming the single tree.

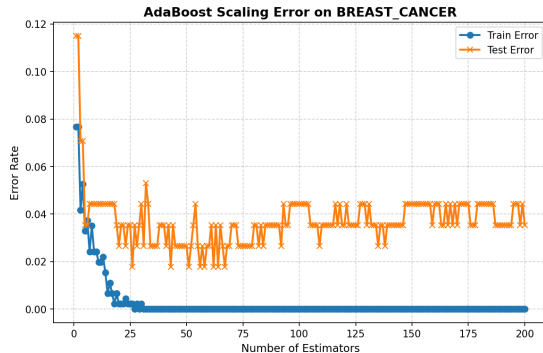
Exp. 2 - AdaBoost Scaling: #Stumps vs. Accuracy

What we measure: Staged train and test accuracy as T (number of stumps) grows from 1 to 200.

Expected behaviour: AdaBoost's additive model converges because each stump contributes a small, targeted correction in hypothesis space. The exponential weight update ensures that already-correct examples receive less attention each round.

Key finding:

- Accuracy saturates around $T \approx 50-80$ on Breast Cancer.
- Train and test curves remain close - no catastrophic overfitting because stumps are maximally regularised (depth-1).



Breast Cancer. The near-zero train/test gap demonstrates that depth-1 stumps are weak enough to avoid memorisation, yet ensemble aggregation achieves strong generalisation.

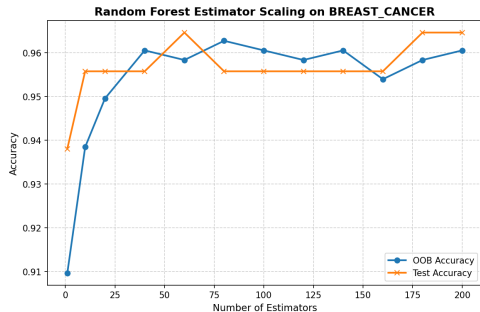
Exp. 3 - Random Forest Scaling: Trees & Depth

(a) Varying #trees (T), fixed max-depth = ∞ :

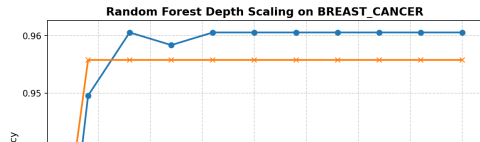
OOB score converges quickly ($T \approx 60$) - consistent with the theoretical result that the ensemble error is bounded by $\bar{\rho}\bar{s}^2$, where $\bar{\rho}$ is mean inter-tree correlation. Once T is large enough, adding more trees offers negligible decorrelation benefit.

(b) Varying max-depth, fixed $T = 100$:

Accuracy plateaus after depth $\approx 4-6$. Deeper trees individually overfit but their errors *cancel* via averaging. This is a direct demonstration that bagging *converts* variance into diversity.



(a) OOB accuracy converges well before $T = 100$.

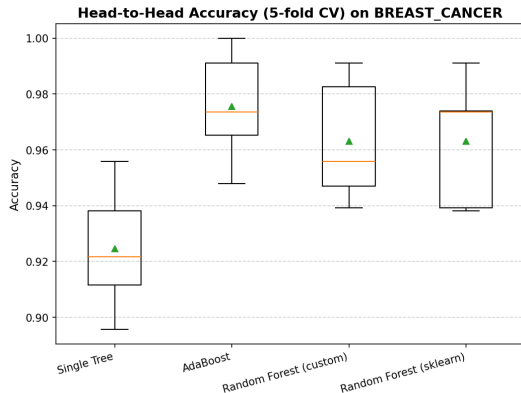


Exp. 4 - Head-to-Head: 5-fold Cross-Validation

Protocol: Stratified 5-fold CV, $T = 100$ estimators, three metrics (Accuracy, Macro-F1, AUC-ROC) on all three datasets.

Key findings:

- **Breast Cancer:** AdaBoost achieves highest accuracy (0.9755 ± 0.0186) - sequential re-weighting exploits the clean, low-noise structure.
- **Adult Income:** Custom RF outperforms AdaBoost on AUC-ROC (0.9169 vs. 0.9073) - bootstrap averaging handles the class imbalance more robustly than weight amplification.
- **MNIST 3-vs-8:** RF dominates (0.9808 ± 0.0035) - random subsampling effectively handles 784 noisy pixel features;



Breast Cancer: both ensembles substantially outperform the single tree. AdaBoost leads narrowly due to dataset's clean, low-variance structure.

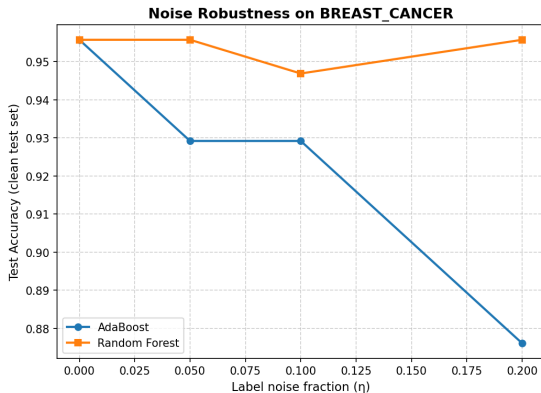
Exp. 5 - Noise Robustness

Protocol: Randomly flip $\eta \in \{0, 0.05, 0.10, 0.20\}$ fraction of training labels. Measure resulting test accuracy.

Mechanistic explanation:

- AdaBoost's weight update *amplifies* focus on misclassified examples. Under label noise, truly mislabelled samples receive exponentially increasing weight - the algorithm learns the corrupted labels rather than the true decision boundary.
- Random Forest's bootstrap sampling dilutes each corrupted label: on average, a flipped label appears in only $\approx 63\%$ of trees, and random feature selection further reduces its influence.

Exception: On Adult Income, AdaBoost degrades



Breast Cancer: AdaBoost loses -7.96% accuracy at $\eta = 0.20$; RF loses 0.00% . The re-weighting mechanism that gives AdaBoost its bias-reduction power becomes its critical weakness under label noise.

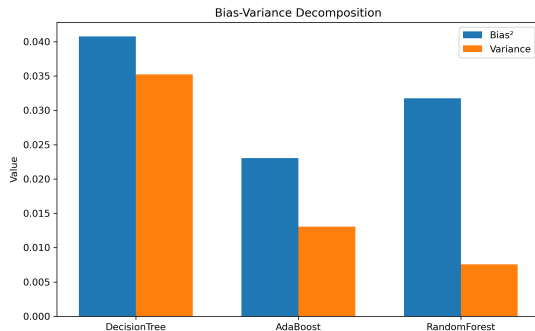
Exp. 6 - Bias-Variance Decomposition

Protocol: $B = 100$ bootstrap replicates of the Breast Cancer training set. Decompose expected error as:

$$\mathbb{E}[\text{err}] \approx \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Model	Bias ²	Variance
Single Tree	0.0408	0.0352
AdaBoost	0.0231	0.0131
Random Forest	0.0317	0.0076

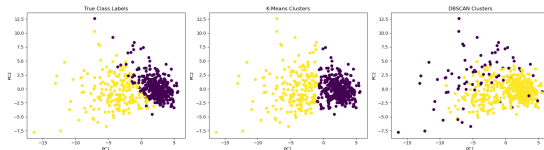
Interpretation: AdaBoost's sequential correction mechanism reduces Bias² by 43% relative to a single tree, at the cost of higher variance. RF halves variance (−78%) while reducing bias by a moderate amount - perfectly consistent with the bagging theory.



Stacked Bias²/Variance bars confirm the fundamental trade-off: AdaBoost targets bias; Random Forest targets variance.

Exp. 7 - Unsupervised Structure

- **PCA Scree:** 7 of 30 principal components explain $\geq 90\%$ of variance - the data lies on a low-dimensional manifold, implying that many raw features are correlated/redundant.
- **K-Means** ($k = 2$, ARI= 0.6536): substantial but imperfect agreement with true labels - PCA space is approximately separable, validating the linear structure of the feature space.
- **DBSCAN** (ARI= 0.0653, 13.2% noise): density-based clustering fails here because Breast Cancer data lacks the sharp density boundaries DBSCAN requires - the two classes form overlapping, ellipsoidal rather than compact clusters.



PCA scatter (PC1 vs PC2) coloured by true label. Clear but imperfect linear separation explains K-Means success and DBSCAN failure.

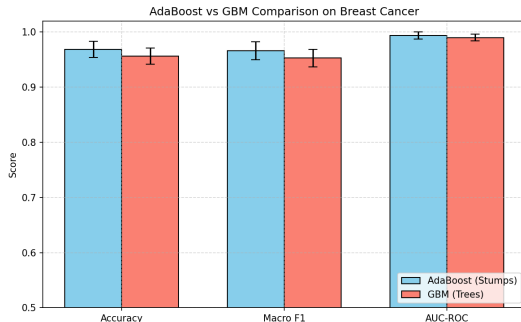
Bonus: GBM vs. AdaBoost & SAMME vs. SAMME.R

GBM vs. AdaBoost (5-fold CV):

- AdaBoost outperforms custom GBM across all datasets.
- GBM uses regression trees on pseudo-residuals (log-loss gradient) - our depth-2 booster may be under-capacity for complex boundaries.
- AdaBoost's *exponential loss* is a tighter upper bound on 0-1 loss than log-loss, which may partly explain the gap.

SAMME vs. SAMME.R (Iris, 80 rounds):

- Both reach 90% test accuracy.
- SAMME.R log-loss: **0.1889** vs. SAMME: 0.6648 - SAMME.R produces *far better calibrated* probability estimates because it updates on soft class probabilities rather than



AdaBoost vs. GBM across CV folds (Breast Cancer).
AdaBoost consistently outperforms.

Discussion: When Does Each Win - and Why?

Boosting wins when:

- Data is clean and low-noise (label corruption is catastrophic).
- The dominant error source is high bias - a single model systematically underfits.
- Dataset is small/structured: sequential weight updates are data-efficient.
- Evidence: Breast Cancer (Bias² reduction -43%, lowest total error 0.0362 vs. 0.0393).

Bagging wins when:

- Labels are noisy: bootstrap averaging dilutes corrupted examples.
- Feature space is high-dimensional: random subsampling decorrelates trees effectively.
- The dominant error source is high variance - individual trees overfit.
- Evidence: MNIST (RF 0.9808 vs. AdaBoost 0.9505); noise robustness (0.00% drop vs. -7.96%).

Neither algorithm uniformly dominates. The correct choice depends on data cleanliness, dimensionality, and whether the dominant error source is bias or variance.

Conclusion

- Implemented CART, AdaBoost (SAMME/SAMME.R), and Random Forest **from scratch in NumPy**; all match sklearn within $\leq 0.25\%$ accuracy, confirming algorithmic correctness.
- **AdaBoost** reduces Bias² more aggressively (-43% vs. single tree) through sequential re-weighting - at the cost of instability under label noise.
- **Random Forest** reduces Variance more dramatically (-78% vs. single tree) through decorrelated bootstrap averaging - and is robust to 20% label corruption.
- Practitioners' rule of thumb: use *boosting* on clean, structured data where underfitting is the bottleneck; use *bagging* when data is noisy, high-dimensional, or class-imbalanced.

Thank you - Questions?